

U1

1. ¿Cuáles son los dos tipos principales de programación? Explique brevemente cada uno de ellos.

La programación imperativa se caracteriza por describir paso a paso las instrucciones que deben ejecutarse para modificar el estado del programa y alcanzar la solución a un problema. Es un enfoque que se centra en cómo se debe realizar una tarea.

La programación declarativa, por otro lado, se centra en que se quiere resolver, describiendo el problema en lugar de las instrucciones específicas para solucionarlo. El programador describe el problema a nivel de usuario, dejando que mecanismos internos se encarguen de inferir la información necesaria para encontrar la solución a partir de esa descripción.

2. Especifique cómo funciona un compilador.

El compilador recibe como entrada el código fuente escrito en un lenguaje de alto nivel (como C o C++) y genera como salida un código objeto o lenguaje máquina, que es el lenguaje que la computadora entiende directamente, listo para ser ejecutado por la computadora.

3. Diga que es un interprete

Un intérprete es un software que recibe un programa en lenguaje de alto nivel, lo analiza y lo ejecuta. Para analizar el programa completo, va traduciendo sentencias de código y ejecutándolas si están bien, así hasta completar el programa origen.

4. ¿Cuál es la diferencia entre la verificación estática y la verificación dinámica de tipos?

La verificación estática se realiza en tiempo de compilación, mientras que, la verificación dinámica se realiza en tiempo de ejecución.

Si la información de tipos se conserva y se verifica en tiempo de ejecución, la verificación es dinámica.

Si la verificación se realiza durante la traducción de un programa entonces es estática.

5. ¿Qué es la conversión explícita?

La conversión explícita, también llamada conversión forzada o cast, es un conjunto de funciones integradas que el programador puede llamar para realizar una conversión de tipo.

6. ¿Qué es un objeto de datos y que atributos tiene? Explique brevemente cada uno de los atributos.

Un objeto de datos es un recipiente que se usa para guardar y recuperar valores de datos.

Atributos:

- Valor: e representa el dato como un patrón de bits en la memoria de la computadora. Es la información que el objeto contiene.
- Tipo: define el conjunto de valores de datos que el objeto puede tomar. El tipo determina las operaciones permitidas sobre el objeto y la forma en que se representa en memoria.

- Identificador: nombre a través del cual se puede referenciar el objeto de datos en tiempo de ejecución. Es el nombre que el programador le asigna para poder acceder a él.
- Localización: son lugares de memoria o direcciones de memoria donde se almacenan los valores.

7. Explique que es un tipo de dato estructurado, ejemplifique.

Los tipos de datos estructurados son objetos de datos contruidos por un agregado de otros objetos de datos llamados componentes. Se los conoce también como estructuras de datos, y pueden ser de tamaño fijo o variable.

Ejemplos: Arreglos, Registros (structs en C), Cadenas de caracteres, Listas, Archivos.

8. Diga que es alcance y apertura de alcance.

El alcance de una variable se refiere al área del programa en el que la variable es visible y puede ser utilizada.

En programación estructurada, el alcance de una variable se determina por el bloque de código en el que se declara.

La apertura de alcance se refiere al punto en el que una variable se vuelve visible y puede ser utilizada en el código. La apertura de alcance de una variable se produce en el punto de su declaración.

9. Defina variable local y externa, indique que alcance tiene cada una.

Una variable local es una variable declarada dentro de una función. Su alcance se limita al bloque de código donde se declara, incluyendo cualquier bloque anidado dentro de ese bloque. Esto significa que solo puede ser accedida desde dentro de la función donde fue definida; no es visible ni accesible desde otras funciones.

Una variable externa o global (en el caso de ser definida antes de la función main), tiene un alcance que se extiende a todas las funciones del programa. Una vez definida fuera de cualquier función, cualquier función puede acceder a ella utilizando su nombre.

10. Explique que es un subprograma y sus partes.

En C se conocen como subprogramas o funciones aquellos trozos de códigos utilizados para dividir un programa con el objetivo de que cada uno realice una tarea determinada para resolver una parte del problema.

El uso de subprogramas permite dividir un programa en un cierto número de componentes más pequeñas, cada una de estas con un propósito específico y determinado.

Un subprograma consta de dos partes principales:

Segmento de código (parte estática): Esta parte contiene el código ejecutable del subprograma, así como las constantes definidas dentro de él. Es la parte que permanece invariable durante las diferentes ejecuciones del subprograma.

Registro de activación (parte dinámica): Esta parte se crea cada vez que se llama al subprograma y se destruye al finalizar su ejecución. Contiene información que varía en cada ejecución, incluyendo:

- Parámetros: Los valores que se pasan al subprograma como entrada.
- Resultados de la función: Los valores que el subprograma devuelve al programa principal.
- Datos locales: Variables declaradas dentro del subprograma, que solo existen durante su ejecución.
- Punto de retorno: La dirección de memoria a la que debe regresar el control del programa una vez que el subprograma ha finalizado.

11. ¿De qué maneras una función puede devolver más de un valor? Explique al menos dos formas.

Dos formas comunes son:

1. Pasaje de parámetros por dirección: en lugar de pasar los valores a la función directamente, se pasan las direcciones de memoria donde se almacenan esos valores (usando el operador & delante del parámetro real de la función en el main). Dentro de la función, se utilizan punteros (*) para acceder a las ubicaciones de memoria y modificar los valores originales.
2. Pasaje de parámetros por referencia: permite que una función acceda y modifique directamente el valor de una variable ubicada en la memoria del programa principal, sin crear una copia de la misma. La función recibe como parámetro un puntero que apunta directamente al espacio de memoria donde se encuentra almacenada la variable y así poder ser modificada. Esto significa que el parámetro formal dentro de la función se convierte en un alias de la variable original, de modo que cualquier cambio realizado en el parámetro formal se refleja directamente en la variable original.

12. Indique como se organiza la memoria en lenguaje C y describa cada parte. Ejemplifique con un gráfico.

La organización de la memoria en C se divide principalmente en tres zonas: almacenamiento estático, pila (stack) y montículo (heap).

1. Almacenamiento estático: en esta área se almacenan los segmentos de códigos de las funciones y otros datos del sistema.
2. Pila (Stack): La pila es un bloque secuencial de memoria. El lenguaje C utiliza la pila para gestionar las llamadas a las funciones. En esta área se almacenan los registros de activación de las funciones.
Cada vez que se llama a una función, se crea un nuevo marco de pila (stack frame) que contiene los registros de activación de la función. Cuando la función termina, su marco de pila se destruye, liberando la memoria ocupada. La pila crece y decrece de forma dinámica durante la ejecución del programa.
3. Montículo (Heap): es un bloque de almacenamiento dentro del cual se asignan o liberan segmentos de memoria. A diferencia de la pila, la memoria en el montículo no se libera automáticamente, por lo que es responsabilidad del programador liberar la memoria usando free.

13. Explique las partes de un registro de activación.

Parámetros: Los valores que se pasan al subprograma como entrada. Estos valores son copiados al registro de activación al momento de la llamada al subprograma.

Resultados de la función: El o los valores que el subprograma devuelve al programa principal. El espacio para almacenar estos resultados se reserva en el registro de activación.

Datos locales: Variables declaradas dentro del subprograma, que solo existen durante su ejecución. Estas variables se almacenan en el registro de activación y su vida útil se limita a la duración de la llamada al subprograma.

Punto de retorno: La dirección de memoria a la que debe regresar el control del programa una vez que el subprograma ha finalizado. Este puntero indica la ubicación en el código donde se reanudará la ejecución después de la llamada al subprograma.

Áreas temporales de almacenamiento: Espacios de memoria utilizados para almacenar valores intermedios durante el cálculo.

14. ¿Qué diferencia hay entre una variable estática y una automática?

Variables Automáticas: Estas variables se declaran dentro de una función y su alcance se limita a esa función. Su tiempo de vida es la duración de la ejecución de la función; se crean cuando la función se invoca y se destruyen cuando la función termina. Se almacenan en la pila, dentro del registro de activación de la función.

Variables Estáticas: Estas variables también pueden declararse dentro de una función, pero a diferencia de las automáticas, su tiempo de vida abarca toda la ejecución del programa. Aunque su alcance es el mismo que el de las variables automáticas (locales a la función), retienen su valor entre llamadas consecutivas a la función. Se almacenan en el área de almacenamiento estático, después del segmento de código de la función.

15. Explique que es y como funciona una tabla de símbolos

Una tabla de símbolos es una estructura de datos. Es como un diccionario de variables, cuya función principal es almacenar información sobre los identificadores utilizados en el código fuente.

Funcionamiento: para cada identificador encontrado en el código, la TS guarda una entrada que contiene, como mínimo, el nombre del identificador. Sin embargo, la información adicional almacenada depende del lenguaje de programación y del compilador.

Cuando se entra en un nuevo bloque de código, las nuevas declaraciones se añaden a la pila, y cuando se sale del bloque, se eliminan las entradas correspondientes de la pila. Esto asegura que el compilador solo acceda a las declaraciones que están actualmente en el alcance.

16. Explique que es un arreglo bidimensional, ejemplifique.

Un arreglo bidimensional, también conocido como matriz o tabla, es una estructura de datos que organiza elementos del mismo tipo en una estructura de filas y columnas. Se puede visualizar como una cuadrícula donde cada elemento se accede mediante dos índices: uno para la fila y otro para la columna.

17. ¿Qué es recursividad y que estructura tiene una función recursiva? Explique brevemente

La recursividad es un proceso mediante el cual se puede definir una función en términos de sí misma. En programación, es una técnica que permite a una función llamarse a sí misma para resolver un problema.

Una función recursiva debe tener dos componentes esenciales para evitar un bucle infinito:

1. Caso base: una condición que, al cumplirse, detiene las llamadas recursivas y devuelve un valor.
2. Caso recursivo: la parte de la función donde se llama a si misma, pero con una entrada modificada que la acerca al caso base en cada llamada.

18. ¿Qué sucede en memoria cuando se ejecuta una función recursiva? Explique brevemente.

Cuando se ejecuta una función recursiva, las sentencias que aparecen después de cada invocación no se resuelven inmediatamente, estas quedan pendientes. Por cada invocación recursiva, se genera y se apila en el stack o pila un registro de activación. Luego, una vez alcanzado el caso base y finalizadas las llamadas recursivas, esos registros de activación se van desapilando en el orden inverso a como fueron apilados, ejecutándose además aquellas sentencias que quedaron pendientes.

19. En que se diferencia la recursión de la iteración

La iteración utiliza estructuras de control de repetición explícitas como for o while para ejecutar un bloque de código repetidamente hasta que se cumpla una condición determinada. En cambio, la recursión se basa en la auto-llamada de una función, donde la función se llama a si misma dentro de su propia definición hasta que se alcanza un caso base que detiene la recursión.

20. ¿Qué sucede para que termine una función recursiva? Ejemplifique y explique su respuesta.

Una función recursiva termina cuando se alcanza un caso base. El caso base es una condición que, al cumplirse, impide que la función se llame a sí misma, deteniendo así la recursión. Sin un caso base, la función se llamaría indefinidamente, resultando en un error de pila (stack overflow).

Ejemplo: función factorial recursiva:

```
int factorial(int i) {  
    if (i == 0) {  
        return 1; // Caso base: factorial de 0 es 1  
    } else {  
        return i * factorial(i - 1); // Llamada recursiva  
    }  
}
```

En este ejemplo el caso base es $i == 0$. Cuando el valor de i llega a 0, la función retorna 1 directamente, sin realizar más llamadas recursivas. Si i no es 0, la función se llama a si misma con el valor $i - 1$, acercándose gradualmente al caso base. Este proceso continua hasta que i finalmente se convierte en 0, momento en el que la recursión se detiene y la función comienza a retornar valores a través de la pila de llamadas.

21. ¿Qué diferencia existe entre el almacenamiento estático y el dinámico?

Almacenamiento estático: La memoria para las variables estáticas se reserva en tiempo de compilación. Esto significa que el compilador determina el espacio de memoria necesario para cada variable antes de que el programa comience a ejecutarse. Este espacio permanece asignado durante toda la ejecución del programa o hasta que la función que contiene la variable deja de ejecutarse, en el caso de variables locales. Su tamaño es fijo y no puede cambiar durante la ejecución.

Almacenamiento dinámico: A diferencia del almacenamiento estático, la memoria para las variables dinámicas se asigna en tiempo de ejecución. Esto significa que el programa solicita memoria al sistema operativo solo cuando la necesita. El tamaño de la memoria asignada puede variar, y puede ser liberada explícitamente por el programa cuando ya no es necesaria.

22. ¿Qué diferencia hay entre “Basura” y “referencia desactivada” al manipular datos dinámicos?

Basura: Se refiere a un bloque de memoria que ya no es accesible por el programa, pero que aún no ha sido liberado por el sistema operativo. Esto ocurre cuando se pierde toda referencia (puntero) a un bloque de memoria previamente asignado dinámicamente, pero la memoria no se libera explícitamente usando free (en C). El espacio ocupado por la basura permanece asignado.

Referencia desactivada o bamboleante: Ocurre cuando se libera la memoria de un objeto dinámico (usando free) antes de que se eliminen todas las referencias (punteros) a él. Esto deja punteros "colgando" que apuntan a una zona de memoria ya liberada. A diferencia de la basura, la referencia desactivada no se refiere a la memoria en sí, sino a la existencia de punteros inválidos que apuntan a una zona de memoria que ya no pertenece al programa.

23. ¿Qué es una “lista” y de qué formas puede ser implementada?

Una lista es una estructura de datos que almacena un conjunto de elementos del mismo tipo, con la característica de poder contener un número indeterminado de elementos y mantener un orden explícito. Es una estructura dinámica, lo que significa que su tamaño puede cambiar durante la ejecución del programa, aumentando y disminuyendo el número de nodos. Todos los elementos son accesibles, y se puede insertar o suprimir un elemento en cualquier posición.

Las listas pueden ser implementadas de varias formas:

- Mediante un arreglo estático: se declara un arreglo de tamaño fijo para almacenar los elementos de la lista. El tamaño del arreglo se define en tiempo de compilación y no puede cambiar durante la ejecución del programa.
- Mediante un arreglo dinámico: se utiliza un puntero a un arreglo, cuyo tamaño se asigna dinámicamente en tiempo de ejecución utilizando la función malloc.
- Mediante punteros (listas enlazadas): cada elemento de la lista se almacena en una estructura (nodo) que contiene, además de la información del elemento, un puntero al siguiente elemento de la lista. El último elemento de la lista contiene un puntero NULL para indicar el final de la lista.

24. Indique cómo funciona la función malloc y free. Explique qué hace el sistema operativo cada vez que se ejecutan.

La función malloc solicita al sistema operativo un bloque de memoria del tamaño especificado en bytes. El sistema operativo, a través de su gestor de memoria, busca un espacio libre de ese tamaño en el montículo y devuelve la dirección de inicio de ese bloque de memoria como un puntero a void. Este puntero genérico debe ser convertido explícitamente al tipo de dato apropiado usando un cast. Si no hay suficiente memoria libre, malloc devuelve NULL. El sistema operativo actualiza su tabla de direcciones de memoria para registrar que ese bloque esta ahora asignado al proceso que realizo la llamada a malloc.

La función free por otro lado, libera el bloque de memoria previamente asignado por malloc. Se le pasa como argumento la dirección de inicio del bloque (el puntero devuelto por malloc). El sistema operativo al recibir la llamada a free, marca el bloque de memoria como disponible en su tablade direcciones de memoria, permitiendo que pueda ser utilizado posteriormente por otras llamadas a malloc.

25. Explique brevemente, con sus palabras y de un modo técnico, las tres formas de inserción en lista.

- Inserción al principio (cabeza): esta es la forma más sencilla. Se crea un nuevo nodo, se hace que su puntero siguiente apunte al nodo que actualmente es el primero de la lista, y luego se actualiza el puntero "cabeza" de la lista para que apunte al nuevo nodo. Esto hace que el nuevo nodo se convierta en el primer elemento de la lista.
- Inserción al final: se recorre la lista hasta encontrar el ultimo nodo. Una vez encontrado, se crea un nuevo nodo, s ele asigna el dato y su puntero siguiente se establece en NULL (para indicar el final de la lista). Finalmente se actualiza el puntero siguiente del ultimo nodo para que apunte al nuevo nodo.
- Inserción al medio: esta forma es mas compleja. Es necesario encontrar primero la posición donde se desea insertar el nuevo nodo. Esto implica recorrer la lista hasta llegar al nodo que precede a la posición de inserción. Una vez encontrada la posición, se crea el nuevo nodo, s ele asigna el dato y se establece su puntero siguiente apuntado al nodo que originalmente ocupaba esa posición. Finalmente, se actualiza el puntero siguiente del nodo anterior para que apunte al nuevo nodo.

26. Explique una forma de clasificación de archivos.

Clasificación de archivos según la dirección del flujo de datos:

- Archivos de entrada: el programa lee datos desde el archivo.
- Archivos de salida: el programa escribe datos en el archivo.
- Archivos de entrada/salida: se pueden leer y escribir datos en el archivo.

27. Explique que es un archivo sin formato.

Es un archivo donde los datos se almacenan directamente en su representación binaria, sin ninguna estructura o formato predefinido impuesto por un programa o sistema específico. A diferencia de los archivos de texto o aquellos con formato específico, un archivo sin formato no tiene metadatos ni información adicional que describa la estructura de los datos.

28. Explique que es un archivo de texto.

Es un tipo de archivo donde cada símbolo o carácter se representa mediante un byte, utilizando codificaciones como ASCII o EBCDIC. En estos archivos, ciertos valores de bytes tienen un significado especial. Si se abre un archivo en modo texto, no será posible leer más allá de ese byte. Estos archivos son secuenciales, con longitud de registro variable, siendo el salto de línea el delimitador de cada registro.

29. Explique que sucede internamente cuando se realiza la apertura de un archivo.

Primero, el SO verifica la existencia del archivo especificado por su nombre. Si el archivo existe y el modo de acceso solicitado es compatible, se crea una entrada en la tabla de archivos abiertos del sistema operativo.

A continuación, se reserva un espacio en la memoria del programa para una estructura FILE, que actúa como un descriptor o manejador del archivo.

Finalmente, el puntero a la estructura FILE se retorna a la función fopen, permitiendo que el programa pueda acceder y manipular el archivo a través de este puntero. Si la apertura falla, la función fopen retorna NULL. El proceso de cierre del archivo, mediante fclose, implica liberar los recursos del SO asociados a la entrada en la tabla de archivos abiertos y la estructura FILE en memoria.

30. Indique de que forma se pueden leer varias componentes de un archivo en una sola lectura.

En C, la función fread permite leer múltiples componentes de un archivo en una sola operación. Fread permite leer un bloque de datos a la vez. La función toma como parámetros: la dirección de memoria donde se almacenarán los datos leídos, el tamaño de cada componente, la cantidad de componentes a leer y el puntero al archivo. El valor retornado por fread indica la cantidad de componentes efectivamente leídas. Esto optimiza el proceso de lectura, reduciendo el número de llamadas al sistema y mejorando el rendimiento.

¿Qué es un lenguaje de programación y cual es su principal función?

Un lenguaje de programación es un sistema notacional para describir computaciones de una manera legible tanto para personas como para máquinas. Su principal función es la construcción de programas que, una vez ejecutados en una computadora, realizan las tareas especificadas en dicho programa.

1. ¿Qué es un paradigma de programación y como se clasifican?

Un paradigma de programación es un método para llevar a cabo cálculos y la forma en que deben estructurarse y organizarse las tareas que debe realizar un programa.

Se clasifican en dos tipos principales:

- Programación Imperativa: se describe paso a paso un conjunto de instrucciones que deben ejecutarse para variar el estado del programa y hallar la solución, es decir, un algoritmo que describe los pasos necesarios para solucionar el problema.
- Programación Declarativa: las sentencias describen el problema que se quiere solucionar; se programa diciendo lo que se quiere resolver, sin especificar las instrucciones necesarias para solucionarlo, las cuales se realizan mediante mecanismos internos de inferencia de información a partir de la descripción realizada.

2. ¿Qué diferencia hay entre un intérprete y un traductor?

Un intérprete es un software que recibe un programa en lenguaje de alto nivel, lo ejecuta línea por línea, si están bien, hasta completar el programa origen, sin generar un programa equivalente en lenguaje de máquina.

Por el otro lado, por traductor se denota a cualquier “procesador de lenguajes” que acepta programas en cierto lenguaje fuente (que puede ser de alto o bajo nivel) como entrada y produce programas funcionalmente equivalentes en otro lenguaje objeto.

¿A que se hace referencia con Simulación de software? (Interpretación)

La simulación de software, también llamada interpretación, hace referencia a un software que recibe un programa en lenguaje de alto nivel, lo analiza y lo ejecuta. Para analizar el programa completo, va traduciendo sentencias de código y ejecutándolas si están bien, así hasta completar el programa origen.

¿Cuáles son las principales características de un lenguaje de programación eficiente?

- Eficiencia de código: el diseño del lenguaje permite que el traductor (compilador o intérprete) genere código ejecutable optimizado, utilizando recursos de manera eficiente.
- Eficiencia de traducción: el lenguaje está diseñado de forma que su código fuente se pueda traducir rápidamente, con un traductor de tamaño razonable.
- Eficiencia de implementación: el lenguaje facilita la creación del propio traductor (compilador o intérprete). Un diseño simple y claro facilita la implementación de un traductor eficiente.
- Eficiencia de programación: el lenguaje facilita la escritura y el mantenimiento de programas. Un lenguaje con una sintaxis clara y concisa, y una buena capacidad de abstracción, contribuye a una mayor eficiencia en el desarrollo de software.

¿Que son las computadoras simuladas por software y para que se utilizan?

Son programas que se ejecutan en una computadora anfitrión para simular el comportamiento de una computadora con un lenguaje de alto nivel. En lugar de traducir directamente un programa de alto nivel a lenguaje máquina, se utiliza un intérprete que analiza y ejecuta el programa de alto nivel sentencia por sentencia. Se utilizan para ejecutar programas en lenguajes de alto nivel en computadoras que no tienen el hardware necesario para ejecutarlos directamente, o para facilitar el desarrollo y la depuración de programas.

Diga cuales son los pasos para derivar un algoritmo. Explique brevemente.

1. Definición del problema: se debe definir claramente el problema a resolver, especificando sus entradas, salidas y restricciones.
2. Descomposición del problema: el problema se divide en subproblemas mas pequeños y manejables. Este proceso se repite recursivamente hasta que los subproblemas sean lo suficientemente simples como para ser resueltos directamente.
3. Diseño de los módulos: para cada subproblema, se diseña un modulo (una parte del programa) que lo resuelve. Estos módulos pueden ser funciones o procedimientos que realizan una tarea específica.
4. Implementación de los módulos: se escribe el código para cada módulo, utilizando un lenguaje de programación adecuado.

5. Integración de los módulos: los módulos se integran para formar el algoritmo completo.
6. Prueba y depuración: el algoritmo se prueba exhaustivamente para verificar su correcto funcionamiento y se corrigen los errores que se encuentren.

¿Cómo se diferencia un lenguaje imperativo de un lenguaje declarativo?

En la programación imperativa, se especifica cómo debe resolver el problema el ordenador mediante una secuencia de instrucciones que modifican el estado de la máquina paso a paso.

Por el contrario, en la programación declarativa, se describe qué resultado se desea obtener, sin especificar el proceso exacto para lograrlo.

¿Qué son los traductores y como funcionan en el proceso de ejecución de un programa?

Los traductores son procesadores de lenguajes que toman un programa escrito en un lenguaje fuente (de alto o bajo nivel) y lo transforman en un programa equivalente en otro lenguaje objeto. Existen dos tipos principales: compiladores e intérpretes.

Un compilador traduce todo el programa fuente a lenguaje máquina de una sola vez, generando un archivo ejecutable que se puede ejecutar posteriormente sin necesidad del compilador.

Un intérprete, en cambio, ejecuta el programa fuente línea por línea, traduciendo y ejecutando cada instrucción a medida que se encuentra. No genera un archivo ejecutable separado; la ejecución depende directamente del intérprete.

¿Qué es la regularidad en un lenguaje de programación y como se relaciona con la eficiencia?

La regularidad en un lenguaje de programación se refiere a la existencia de pocas restricciones en el uso de sus constructores, pocas interacciones inusuales entre ellos y una menor cantidad de comportamientos inesperados. Se busca que el lenguaje sea predecible y consistente en su funcionamiento.

La regularidad está relacionada con la eficiencia, tanto en la escritura del código como en la eficiencia de la traducción y ejecución. Un lenguaje regular facilita la escritura de programas más claros, con menos errores y más fáciles de mantener. Un lenguaje regular también suele ser más fácil de traducir (compilar o interpretar), lo que resulta en un proceso de traducción más rápido y un código generado más eficiente.

Explique que es un compilador y en que se diferencia de un intérprete.

Un compilador es un software que traduce un programa escrito en lenguaje de alto nivel a un lenguaje de máquina, que es el lenguaje que la computadora entiende directamente. Este proceso de traducción se realiza completamente antes de que el programa se ejecute. El resultado es un archivo ejecutable que contiene el código en lenguaje máquina.

La principal diferencia radica en que el compilador produce un programa ejecutable independiente, mientras que el intérprete necesita estar presente cada vez que se ejecuta el programa. El intérprete no genera un archivo ejecutable separado.

U2

1. ¿Qué es el tiempo de vida de una variable y como se relaciona con su alcance?

El tiempo de vida de una variable se refiere al periodo durante el cual la variable existe y puede utilizarse para almacenar datos. Este tiempo se extiende desde el momento en que se crea la variable hasta el momento en que se destruye.

El alcance de una variable, por otro lado, define la región del programa donde la variable es accesible. Una variable puede tener un alcance amplio, pero un tiempo de vida corto. Recíprocamente, una variable puede tener alcance limitado, pero un tiempo de vida largo.

2. ¿Qué son los tipos de datos estructurados, cuáles son los más utilizados y como se clasifican?

-

¿Qué es un arreglo multidimensional?

Un arreglo multidimensional es una estructura de datos que extiende el concepto de un arreglo unidimensional a mas de una dimensión. En esencia, es un arreglo de arreglos.

¿Cómo se clasifica un objeto de dato? Ejemplifique

- Definidos por el programador: Estos son manipulados directamente por el programador a través de declaraciones y sentencias. Ejemplos de estos son variables, constantes y arreglos.
- Definidos por el sistema: estos se construyen y mantienen durante la ejecución del programa sin intervención directa del programador. Ejemplos incluyen pilas de almacenamiento en tiempo de ejecución, registros de activación, etc.

Diga que es la conversión de tipos y explique como se clasifican

La conversión de tipos es una operación que toma un objeto de un tipo y produce el objeto de datos correspondiente de un tipo distinto. Se pueden clasificar en dos tipos principales:

- Conversiones implícitas o coerciones: estas conversiones son invocadas automáticamente por el compilador en ciertos casos de discordancia de tipos entre operandos en una expresión. El principio básico que las rige es no perder información.
- Conversiones explícitas: estas conversiones son realizadas por el programador mediante el uso de operadores de conversión de tipos o funciones específicas del lenguaje (inferencia a partir de la descripción de coerciones implícitas). El programador controla explícitamente la conversión, permitiendo mayor control sobre el proceso y la posibilidad de manejar situaciones donde la conversión podría resultar en pérdida de información.

¿Qué es el enlace (ligadura) y el tiempo de enlace en la programación procedural?

El enlace o ligadura se refiere a la conexión entre un elemento del programa (como un nombre de variable o una función) y una característica o propiedad particular. Esto implica la asignación de una propiedad específica dentro de un conjunto de propiedades posibles a ese elemento.

El tiempo de enlace o tiempo de ligadura es el momento en la ejecución del programa en el que se realiza este enlace. Si el enlace se realiza antes de la ejecución, se denomina enlace temprano o ligadura estática. Por el contrario, si el enlace se realiza en tiempo de ejecución, se llama enlace tardío o ligadura dinámica.

¿Cuáles son los tipos de datos elementales más comunes en lenguaje C?

- Enteros
- Números reales de punto flotante
- Enumeraciones
- Booleanos
- Caracteres
- Punteros

¿Qué es el valor l y valor r de una variable?

En el contexto de la asignación de variables en programación, valor l (lvalue) se refiere a la localización de memoria de una variable, es decir, la dirección de memoria donde se almacena el valor de la variable. Mientras que el valor r (rvalue) representa el valor que se encuentra almacenado en esa localización de memoria.

Explique a que hacen referencia las siguientes notaciones $b[i]$ y $*(b+i)$.

Las notaciones $b[i]$ y $*(b+i)$ son equivalentes y hacen referencia al elemento en la posición i de un arreglo b . $b[i]$ es la forma estándar de acceder a un elemento de un arreglo utilizando la notación de corchetes. $*(b+i)$ utiliza aritmética de punteros: b representa la dirección base del arreglo (la dirección del primer elemento), i es el desplazamiento o número de posiciones desde el primer elemento, y el operador $*$ accede al valor almacenado en la dirección calculada.

U3

1. Explique que sucede con una función (subprograma) en tiempo de compilación y en tiempo de ejecución

En tiempo de compilación: el compilador analiza la declaración y la definición de la función, verificando su sintaxis y semántica. El compilador genera el código objeto correspondiente al cuerpo de la función, creando un segmento de código que contiene las instrucciones a ejecutar. Este segmento de código es la parte estática de la función, compuesta por constantes y código ejecutable. La información sobre la función, como su nombre, tipo de retorno, y parámetros, se registra en la tabla de símbolos. La definición de la función da como resultado una plantilla, a partir de la cual se crearán activaciones particulares cada vez que se ejecute la función.

En tiempo de ejecución, cuando la función es llamada, se crea una instancia de la plantilla de la función, es decir, se crea un registro de activación. Este registro de activación es la parte dinámica de la función y contiene información específica de esa llamada, incluyendo parámetros pasados, variables locales, resultado de la función y dirección de retorno. El registro de activación se suele almacenar en la pila. La ejecución salta al segmento de código generado en tiempo de compilación, y se procesan las instrucciones del cuerpo de la función utilizando la información del registro de activación. Una vez que la función termina su ejecución, su registro de activación se elimina de la pila y la ejecución continúa en la dirección de retorno. Las variables locales, que forman parte del registro de activación, dejan de existir al finalizar la ejecución de la función.

2. ¿Qué es la traducción de un programa fuente y como se genera la tabla de símbolos (TS)?

La traducción de un programa fuente es el proceso de convertir el código escrito en un lenguaje de alto nivel a código máquina que la computadora pueda ejecutar directamente.

La generación de la TS se realiza de forma incremental durante la compilación. A medida que el compilador analiza el código fuente, va añadiendo nuevas entradas a la TS para cada identificador encontrado. Al entrar a un nuevo bloque de código, se añaden las nuevas declaraciones a la TS. Al salir del bloque, las entradas correspondientes se eliminan de la pila.

3. ¿Qué es el pasaje de parámetros en programación procedural?

Es el mecanismo mediante el cual una función recibe información de la parte del programa que la llama. Esta información se proporciona a través de parámetros, que son variables que se declaran en la definición de la función (parámetros formales) y a las que se les asignan valores al momento de llamar a la función (parámetros reales).

4. ¿Qué es el pasaje por dirección? ¿Cómo se relaciona con la memoria del programa?

El pasaje por dirección es un tipo de pasaje de parámetros que consiste en el paso por valor de una dirección, por ello puede usarse para cambiar el contenido de la memoria apuntada por ese puntero. El pasaje de parámetros por dirección permite retornar más de un resultado desde la función.

Explique que es la definición, invocación y activación de subprogramas.

Definición: la definición de un subprograma es una descripción estática del mismo, que contiene información sobre su estructura y comportamiento. Esta información, disponible en tiempo de traducción, incluye el tipo de datos de los parámetros de entrada, las variables locales, el tipo de dato del valor de retorno, y el código del cuerpo del subprograma. Esencialmente, la definición crea una plantilla para la creación de activaciones en tiempo de ejecución. Esta plantilla se divide en una parte estática y una parte dinámica.

Invocación: es la acción de ejecutar el subprograma. Se realiza mediante una instrucción que especifica el nombre del subprograma y los argumentos (o parámetros) que se le pasan como entrada.

Activación: es la instancia de ejecución del subprograma. Cuando un subprograma es invocado, se crea una activación. Esta activación incluye un registro de activación, que es una estructura de datos que contiene la información necesaria para la ejecución del subprograma, incluyendo los parámetros, las variables locales, los valores de retorno, el punto de retorno y áreas temporales de almacenamiento.

Indique al menos dos diferencias entre pasaje por valor y pasaje por referencia.

1. En el pasaje por valor, se crea una copia del valor del parámetro real y se pasa a la función. Esto implica una duplicación del área de memoria. En cambio, en el pasaje por referencia, se pasa la dirección de memoria del parámetro real, no una copia de su valor. No hay duplicación de memoria.
2. En el pasaje por valor, cualquier modificación al parámetro formal dentro de la función no afecta al parámetro real. El parámetro real permanece inalterado. En el pasaje por referencia, cualquier cambio realizado al parámetro formal dentro de la función se refleja en el parámetro real, ya que ambos apuntan a la misma ubicación de memoria.

¿Cuál es la diferencia entre una función y un procedimiento?

Una función, además de realizar una tarea específica, retorna un valor como resultado de su ejecución. Este valor puede ser utilizado por la parte del programa que llamo a la función. En cambio, un procedimiento realiza una tarea, pero no devuelve ningún valor. Su propósito es modificar el estado del programa o realizar una acción, sin producir un resultado directo que pueda ser asignado a una variable.

¿Qué es una tabla de símbolos y que operaciones realiza?

Una tabla de símbolos (TS) es una estructura de datos creada durante la traducción de un programa fuente. Actúa como un diccionario que almacena información sobre los identificadores (nombres de variables, funciones, etc.) declarados en el programa.

Las operaciones principales que realiza una TS son:

- Inserción: Agrega un nuevo identificador a la tabla junto con sus atributos, cuando el traductor encuentra una declaración.
- Búsqueda: Busca un identificador en la tabla para verificar si ya ha sido declarado o para recuperar sus atributos.
- Cancelación (o eliminación): Elimina un identificador de la tabla, generalmente al salir de un bloque de código donde se declaró un identificador local.
- SET (activar un bloque): Se utiliza cuando el compilador detecta el comienzo de un bloque o función, permitiendo la declaración de identificadores locales o automáticos.
- RESET (desactivar un bloque): Se utiliza al final de un bloque o función para eliminar las ligaduras de los identificadores locales.

Explique cómo se activa (que sucede internamente) un registro de activación para la ejecución de una función.

Primero, se necesita una plantilla que define la estructura del registro de activación para esa función específica. Esta plantilla se divide en dos partes: una estática (segmento de código con constantes y código ejecutable) y una dinámica. La parte estática se almacena en memoria estática, mientras que la parte dinámica se crea en la pila cada vez que se llama a la función.

Cuando se invoca una función, se crea un nuevo registro de activación en la pila [doc_1, doc_3]. Este registro contiene información esencial para la ejecución de la función, incluyendo: parámetros, resultados de la función, datos locales, el punto de retorno y áreas temporales de almacenamiento [doc_3, doc_5]. El tamaño y la estructura de este registro se determinan en tiempo de compilación [3]. La información de los parámetros se copia desde el punto de llamada en el registro de activación [3].

Diga que es un registro de activación y explique brevemente cada una de sus partes.

Un registro de activación es una estructura de datos que se crea cada vez que se invoca un subprograma (función o procedimiento). Contiene toda la información necesaria para la ejecución de ese subprograma. Su estructura es fija y se determina en tiempo de traducción por el compilador. Para ahorrar memoria se divide en una parte estática (segmento de código) y una parte dinámica (registro de activación).

Las partes de un registro de activación son:

- Almacenamiento estático: en esta área se almacenan los segmentos de códigos de las funciones y otros datos del sistema.

- Pila (stack): es un bloque secuencial de memoria que se asigna en tiempo de ejecución. La pila se utiliza en lenguaje C para gestionar las llamadas a las funciones. En esta área se almacenan los registros de activación de las funciones. Cuando se invoca una función, el registro de activación correspondiente se almacena en la pila, una vez finalizada la ejecución, el registro se desapila.
- Parámetros: los valores que se pasan a la función como argumentos durante su llamada.
- Resultado de la función: el o los valores que la función devuelve al finalizar su ejecución.
- Variables locales: variables declaradas dentro de la función, que solo son accesibles desde dentro de ella.
- Dirección de retorno: la dirección de memoria a la que debe regresar el control de ejecución una vez que la función termina.
- Áreas temporales de almacenamiento: espacios de memoria utilizados para realizar cálculos intermedios durante la ejecución de la función.
- Montículo: bloque de almacenamiento dentro del cual se asignan o liberan segmentos de memoria.

U4

1. ¿Qué es la recursividad en programación y como se utiliza en lenguaje C?

La recursividad en programación es una técnica donde una función se llama a si misma dentro de su propia definición. Esto permite resolver problemas que se pueden descomponer en subproblemas más pequeños del mismo tipo. En esencia, una función recursiva se define en términos de si misma, repitiendo una operación hasta alcanzar un caso base que detiene la recursión.

En lenguaje C, una función recursiva se implementa declarando una función que contiene una llamada a si misma. La gestión de memoria se realiza a través de la pila, donde se almacenan las variables locales y los parámetros de cada llamada recursiva.

¿Qué diferencia hay entre recursión e iteración? Señala al menos tres.

1. La iteración utiliza estructuras de repetición explícitas como for o while para repetir un bloque de código. La recursión, por otro lado, consigue la repetición mediante llamadas repetidas a la misma función.
2. La iteración se basa en una estructura de repetición, mientras que la recursión utiliza una estructura de selección (generalmente un if o switch) para definir el caso base y el caso recursivo.
3. La iteración termina cuando falla la condición de continuación del ciclo. La recursión termina cuando se alcanza un caso base, una condición que detiene las llamadas recursivas.

Indique y explique las partes de una función recursiva. - ¿Qué estructura tiene una función recursiva? Explique brevemente sus partes.

Una función recursiva se compone de dos partes esenciales:

1. Caso base: es la condición que determina cuando la función deja de llamarse a si misma. Sin un caso base, la función entraría en una recursión infinita, consumiendo recursos hasta que el programa se bloquee.

2. Caso recursivo: esta parte de la función contiene la llamada recursiva a sí misma. Define como la función se aproxima al caso base con cada llamada recursiva.

U5

Unidad 1

1. ¿Cuáles son los dos tipos principales de programación? Explique brevemente cada uno de ellos.

- Imperativa: se describe paso a paso un conjunto de instrucciones que deben ejecutarse para variar el estado del programa y hallar la solución, es decir, un algoritmo en el que se describen los pasos necesarios para solucionar un problema.
- Declarativa: lo que hacen las sentencias que se utilizan es describir el problema que se quiere solucionar, se programa diciendo lo que se quiere resolver a nivel de usuario, pero no las instrucciones necesarias para solucionarlo. Esto último se realizará mediante mecanismos internos de inferencia de información a partir de la descripción realizada.

2. Especifique cómo funciona un compilador

El compilador es un programa que recibe como datos de entrada el código fuente de un programa escrito por un programador, y genera como salida un conjunto de instrucciones escritas en el lenguaje binario de la computadora donde se van a ejecutar. El proceso se realiza en varias fases:

- a) Análisis léxico: el compilador lee el código fuente carácter por carácter y lo divide en unidades significativas llamadas tokens.
- b) Análisis sintáctico: el compilador verifica la estructura gramatical del código fuente, es decir, si las instrucciones están organizadas correctamente.
- c) Análisis semántico: el compilador verifica el significado del código fuente, es decir, si las instrucciones tienen sentido y si las variables se utilizan correctamente.
- d) Generación de código intermedio: el compilador genera un código intermedio que es independiente de la arquitectura de la computadora.
- e) Optimización de código: el compilador optimiza el código intermedio para que se ejecute más rápido y eficientemente.
- f) Generación de código maquina: el compilador genera el código maquina que se puede ejecutar directamente por la computadora.

3. Diga que es un interprete

Un intérprete es un software que recibe un programa en lenguaje de alto nivel, lo analiza y lo ejecuta. Para analizar el programa completo, va traduciendo sentencias de código y ejecutándolas si están bien, así hasta completar el programa origen.

Unidad 2

1. ¿Cuál es la diferencia entre la verificación estática y la verificación dinámica de tipos?

La verificación dinámica de tipos se realiza en tiempo de ejecución, mientras que la verificación estática de tipos se realiza en tiempo de compilación.

2. ¿Qué es la conversión explícita?

La conversión explícita o forzada es un conjunto de funciones integradas que el programador puede llamar para realizar una conversión de tipos.

3. ¿Qué es un objeto de datos y que atributos tiene? Explique brevemente cada uno de los atributos.

Un objeto de datos es un recipiente que se usa para guardar y recuperar valores de datos.

Atributos

Valor: se representa como un patrón de bits en el almacenamiento de la computadora subyacente.

Tipo: conjunto de valores de datos que el objeto puede tomar.

Identificador: nombre a través del cual se puede referenciar el objeto de datos en tiempo de ejecución. Este nombre se construye a partir de un conjunto de caracteres; dependiendo del lenguaje puede haber restricciones.

Localización: son lugares de memoria o direcciones de memoria donde se almacenan los valores.

4. Explique que es tipo de dato estructurado, ejemplifique.

Un objeto de datos que esta construido por un agregado de otros objetos de datos, llamados componentes, se conoce como un objeto de datos estructurado o una estructura de datos. Algunos los define el programador y otros los define el sistema durante la ejecución del programa.

Ejemplos de estructuras de datos: arreglos, registros, cadenas de caracteres, listas, conjuntos y archivos.

5. ¿Cómo se clasifica un objeto de dato? Ejemplifique

Elemental: si el objeto contiene un valor de datos que se manipula siempre como una unidad. Ejemplo: un entero, un real

Estructurado: si un objeto de datos es agregado de otros objetos de datos. Ejemplo: un arreglo, un registro.

6. ¿Qué es el valor l y valor r de una variable?

El valor l de una variable se refiere a la localización de la variable en memoria, es decir, la dirección de memoria donde se almacena el valor de la variable. El valor r de una variable se refiere al valor que contiene dicha variable.

7. Explique a que hacen referencia las siguientes notaciones $b[i]$ y $*(b+i)$

Unidad 3

1. Diga que es alcance y apertura de alcance

El alcance de un vinculo se refiere a la región del programa donde este vinculo se mantiene. En lenguaje C, el alcance de un vinculo queda limitado al bloque donde aparece la declaración asociada y a los bloques contenidos en el interior. Este alcance se llama alcance léxico. La apertura de alcance se refiere al punto a partir del cual el vinculo se vuelve accesible dentro del programa. En lenguaje C, la apertura de alcance se extiende desde el punto siguiente a la declaración hasta el final del bloque en el cual se encuentra.

2. Defina variable local y externa, indique que alcance tiene cada una.

Una variable local es una variable que se declara dentro de una función y solo es accesible dentro de esa función. Su alcance se limita al bloque de código donde se declara, incluyendo los bloques anidados dentro de él. Una variable externa, por otro lado, se declara fuera de cualquier función y es accesible desde cualquier parte del programa. Su alcance se extiende a todo el archivo fuente donde se declara.